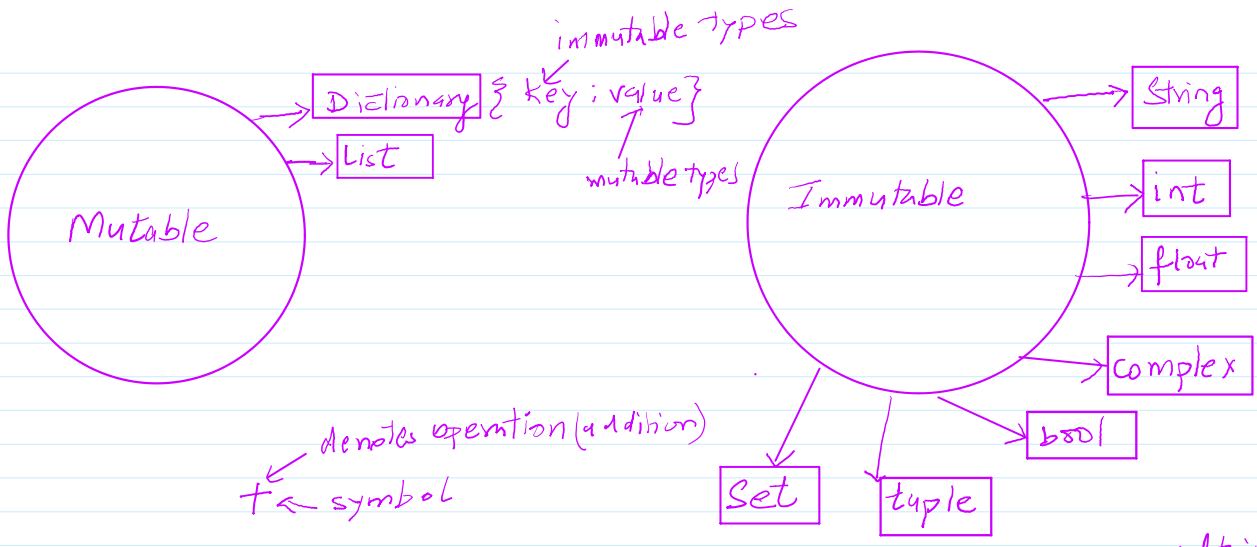
BASICS OF PYTHON PART-2

10 August 2024 16:38



Operators

An operator is a symbol which denotes a type of operation, it may be arithmetic or logical operations.

a > b

result is either True or false

logical operations

Arithmetic operators in Python:

+, -, *, /, //(Quotient or lower division), %(modulus or remainder), **(exponential)

r = a + b

arithmetic operation.

'+' is used to concatenate or join two string values.

```
>>> fn = 'Jiya'
>>> ln = 'Gaur'
>>> nm = fn + ' ' + ln
>>> nm
'Jiya Gaur'
```

a = 100
b = 19.
r = a % b

19) 100 (5 ← quotient
−95
5 ← remainder

'*' if one variable is of type <str> and other data is <int> then it will perform repetition.

```
>>> fn
'Jiya'
>>> fn * 5
'JiyaJiyaJiyaJiyaJiya'
>>> 5 * fn
'JiyaJiyaJiyaJiyaJiya'
```

1×1 = 1
2×2 = 4
3×3 = 9
4×4 = 16

30×30 = 900

Relational operators in Python

≠ ← Maths

!= ← computer

universal not

Operator	Operation	Description	Examples

==	Equality comparison	If the values of two operands are equal, then the condition is True. Otherwise it is False.	>>> a
			100
			>>> b
			19
			>>> a == b
			False
			>>> a == 100
			True
			>>>

() : parenthesis.

			>>>

!=	Not equal to	So if values of two operands are not equal, then condition is True. Otherwise it is False.	>>> a != b
			True
			>>> a != 100
			False

>	Greater than	If the value of the left side operand is greater than the value of right side operand, then condition is True. Otherwise it is False.	>>> a > b
			True
			>>> b > a
			False

<	Less than	If the value of the left side operand is less than the value of right side operand, then condition is True. Otherwise it is False.	>>> a = 100
			>>> b = 23
			>>> a < b
			False
			>>> b < a
			True

>=	Greater than or equal to	If the value of the left side operand is greater than or equal to the value of right side operand, then condition is True. Otherwise it is False.	>>> a >= 100
			True
			>>> a >= b
			True
			>>> b >= a
			False

<=	Less than or equal to	If the value of the left side operand is less than or equal to the value of right side operand, then condition is True. Otherwise it is False.	>>> a <= 100
			True
			>>> a <= b
			False
			>>> b <= a
			True

() : parenthesis.
[] : Brackets.
{} : Braces.

Assignment operators

Operator	Description	Example

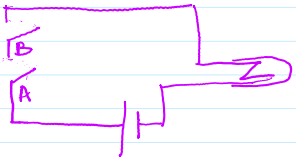
=	Assigns value from right side operand to left side operand.	>>> a = 100
		>>> b = 23
		>>> x = a
		>>> x
		100

+=	It adds the value of right side operand to the left side operand and assigns the result to the left side operand.
	a += b is same as a = a + b
		>>> a
		100
		>>> b
		23
		>>> a += b
		>>> a
		123

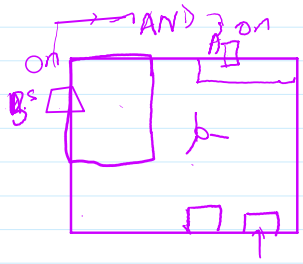
-=	It subtracts the value of right side operand to the left side operand and assigns the result to the left side operand. a -= b is same as a = a - b	
*=	It multiplies the value of right side operand to the left side operand and assigns the result to the left side operand. a *= b is same as a = a * b	
/=	It divides the value of right side	

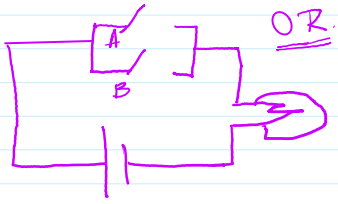
	operand to the left side operand and assigns the result to the left side operand. a /= b is same as a = a / b	
//=	It lower division(Quotient) the value of right side operand to the left side operand and assigns the result to the left side operand. a //= b is same as a = a // b	
%=	It divides and gives the remainder of the value of right side operand to the left side operand and assigns the result to the left side operand. a %= b is same as a = a % b	
**=	It gives the exponential value of right side operand to the left side operand and assigns the result to the left side operand. a **= b is same as a = a ** b	

Logical Operators



A is closed and B is closed then Led will glow.





Operator	Operation	Description	Example

and	Logical AND	If both the operands are True, then condition becomes True.	>>> True and True
			True
			>>> True and False
			False
			>>> False and True
			False
			>>> False and False
			False
			>>>

or	Logical OR	If any of the two operands are True, then condition becomes True.	>>> True or True
			True
			>>> True or False
			True
			>>> False or True
			True
			>>> False or False
			False

not	Logical NOT	Used to reverse the logical state of its operand.	>>> not(True)
			False
			>>> not(False)
			True

Membership Operators

Membership operators are used to check if a value is a member of the given sequence or not.

Operator	Description	Example

in	Returns true if the variable or value is found in the specified sequence and false otherwise.	>>> a = [1, 2, 3, 4, 5]
		>>> 2 in a
		True
		>>> -2 in a
		False
		>>> name = 'Jiya'
		>>> 'y' in name
		True
		>>> 'x' in name

		False >>> type(name) <class 'str'>

not in	Returns True if the variable or value is not found in the specified sequence and False otherwise.	>>> a
		[1, 2, 3, 4, 5]
		>>> -2 not in a
		True
		>>> 2 not in a
		False
		>>> name
		'Jiya'
		>>> 'y' not in name
		False
		>>> 'j' not in name
		True
		>>> 'J' not in name
		False

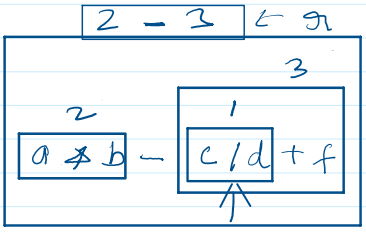
Expressions

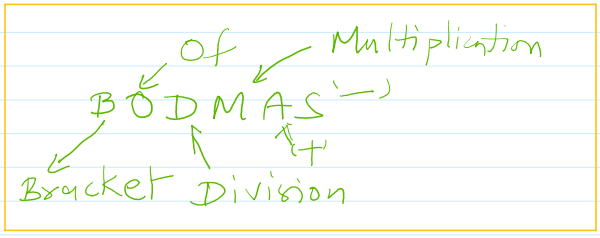
An expression is defined as a combination of constants, variables and operators. And expression always evaluates to a value. A value, or a stand-alone variable, is also considered as an expression. But a stand-alone operator is not an expression.

For example:

```
100
num
3.0 + 4.14
x + 3.0
x * y*(a-b)
"Python"*"Programming" not a valid expression
```

← valid expression.





Precedence of Operators (order of solving expression)

D, M
A, S

$15 * 3 - 20/5 + 6$

$45 - 4 + 6 = 47$

operands

+3 or 3

-3

are unary operator

Evaluation of the expression. is based on precedence of operators. When an expression contains different kinds of operators, precedence determines which operator should be applied first. Higher precedence operator is evaluated before the lower precedence operator Most of the operators studied till now are binary operators. Binary operators are with two operands. The unary operator need only one operand, and they have a higher precedence than the binary operators. The minus, as well as plus operator, can act as both unary and Binary operators. But **'not'** is a unary logical operator.

Precedence of all operators in Python

Order of precedence	Operators	Description
1	**	Exponentiation. (raised to the power)
2	~,+,-	Complement, unary plus, and unary minus
3	*, /, %, //	Multiply, divide, remainder, and floor division(quotient)
4	+, -	Addition and Subtraction
5	<, <=, >, >=	Relational operators
6	==, !=	Equality operators
7	=, %=, /=, //=, -=, +=, *=, **=	Assignment operators
8	is, is not	Identity operators
9	in, not in	Membership operator
10	not, or, and	Logical operators

Note:

a. **Parentheses can be used to override the precedence of operators. The expression within parenthesis'()' is evaluated first.**
b. **For operators with equal precedence. The expression is evaluated from left to right.**

$20-30-40$
$= -10-40$
$= -50$

$20-30+40$
$= -10+40 = 30$

$20-(30+40)$
$= 20-70 = -50$

Statement

In Python, a statement is a unit of code that the Python interpreter can execute.
For example:

```
>>> x = 5
>>> c = x**3
>>> c
125
>>> print('Cube =',c)
Cube = 125
```

Input and Output

Sometimes a program needs to interact with the users to get some input data or information from the end user and process it to give the desired outputs. in python we have the **'input()'** function for taking the user input in the form of string(str). The input function prompts the user to enter data. It accepts all input as a string. The user may enter a number or a string, but the input function treats them as a string only. The syntax for input() function is:

```
input([prompt])
```

Prompt Is the string we may like to display on the screen prior to taking the input and it is optional. When a promise is specified, first it is displayed on the. screen after which the user can enter data. The **input()** takes exactly what is typed from the keyboard, converts it into a string and assign it to the variable on the left hand side of the assignment operator(=). Entering data for the input function is terminated by pressing the enter key.

```
>>> fname = input('Enter your first name: ')
Enter your first name: Shiv
>>> fname
'Shiv'
>>> lname = input('Enter your last name: ')
Enter your last name: Mishra
>>> lname
'Mishra'
>>> type(lname)
```

```
<class 'str'>
>>> type(fname)
<class 'str'>
```

Inserting int or float data in a variable

```
>>> x = int(input('Enter any number: '))
Enter any number: 34
>>> x
34
>>> fx = float(input('Enter any number: '))
Enter any number: 23.71
>>> type(x)
<class 'int'>
>>> type(fx)
<class 'float'>
```

Python uses the print() function to output data to standard output device, the screen or the terminal.

The print() function evaluates the expression before displaying it on the screen. The print() function outputs a complete line and then moves to the next line for subsequent output. The syntax for print() function is:

print(value [, …, sep = ' ', end = '\n'])

- **sep:** The optional parameter sep is a separator between the output values. We can use a character integer or a string as a separator. The default separator is space. For example:

  ```
  >>> x = int(input('Enter any number: '))
  Enter any number: 34
  >>> x
  34
  >>> fx = float(input('Enter any number: '))
  Enter any number: 23.71
  >>> type(x)
  <class 'int'>
  >>> type(fx)
  <class 'float'>
  ```

- **end:** This is also optional and it allows us to specify any string to be appended after the last value. The default is a new line'\n'.
  For example:

  ```
  print('A')
  print('B')
  print('A', end = '')
  print('B', end = '')
  ```

  Output:

  ```
  A
  B
  AB
  ```

Type Conversion

As and when required we can change the data type of a variable in python from 1 type to another. Such data type conversion can happen in two ways, either (Explicitly) or forced. when the programmer specifies for the interpreter to convert a data type to another type or implicitly when the interpreter understands such a need by itself and does the type conversion automatically.

Explicit Conversion

Explicit conversion also called typecasting happens when the data type conversion takes place because the programmer forced it in the program. The general form of an explicit data type conversion is:

(new_datatype)(expression)

With explicit type conversion, there is a risk of loss of information, since we are forcing an expression to be a specific type. For example,

```
>>> x = 20.67
>>> type(x)
<class 'float'>
>>> y = int(x)
>>> type(y)
<class 'int'>
>>> y
20
```

Following are some of the functions in python that are used for explicitly converting an expression or a variable to a different type.

Explicit type conversion functions in python.

Function	Description
int(x)	Converts X to an integer.
float(x)	Converts X to a floating point number.
str(x)	Converts X to a string representation.
chr(x)	Converts X to a character.

```
>>> x = 65
>>> y = chr(x)
>>> y
'A'
>>> n = ord('Z')
>>> n
90
>>> m = chr(n)
>>> m
'Z'
>>> sa = chr(97)
>>> sa
'a'
>>> sz = chr(122)
>>> sz
'z'
>>>
```

character values

Ordinal

Unicode:

```
>>> print('\u091C\u093F\u092F\u093E')
```

use

hexa-decimal digits. or values.

→ Decimal Number (0, 1, 2, 3, ... 9)

→ Binary Number (0, 1)

→ Octal Number System (0, 1, 2, 3, 4, 5, 6, 7).

→ Hexa-decimal number System (16 symbols) (0 to 9, A, B, C, D, E, F)

Implicit conversion: Implicit conversion, also known as coercion, Happens when data type conversion is done automatically by python and is not instructed by the programmer.

```
>>> x
65
>>> f = x + 1.0
>>> type(f)
<class 'float'>
>>> f
66.0
```

int

float

int + float = float (Implicitly).

Debugging

Programmer can make mistakes While writing a program and hence the program may not execute or may generate wrong output. The process of identifying and removing such mistakes are also known as bug or errors from the program is called **debugging**. Errors occurring in program can be categorized as:

i. Syntax errors
ii. Logical errors
iii. Runtime errors

Syntax Errors

Like other programming languages, python has its own rules that determine its syntax. The interpreter interprets the statements only if it is syntactically or as per the rules of Python correct. If any syntax error is present the interpreter shows error messages and stops the execution there.

For example:

```
>>> print 'hello'
  File "<stdin>", line 1
    print 'hello'
    ^^^^^^^^^^^^^
SyntaxError: Missing parentheses in call to 'print'. Did you mean print(...)?
```

Logical Errors

A logical error is a bug in the program that causes it to behave incorrectly. A logical error produces an undesired output, but without abrupt termination of the execution of the program. Since the program interprets successfully, even when the logical errors are present in it, it is sometimes difficult to identify these errors. The only evidence to the existence of logical errors is the wrong output. While working backwards from the output of the program, 1 can identify what went wrong.

Logical errors are also called semantic errors, as they occur when the meaning of the program is not correct.

Runtime Error

By runtime error causes abnormal termination of a program while it is executing. Runtime error is when the statement is correct syntactically, but the interpreter cannot execute it. Runtime errors do not appear until after the program starts running or executing. For example, we have a statement having division operation in a program. By mistake, if the denominator entered is zero, then it will give a runtime error, like "division by 0".

```python
x = float(input('Enter numerator: '))
y = float(input('Enter denominator: '))
r = x / y
print('r =',r)
```

```
Enter numerator: 18
Enter denominator: 0
Traceback (most recent call last):
  File "c:\Users\bluej\Documents\Students\I
", line 4, in <module>
    r = x / y
        ~~^~~
ZeroDivisionError: float division by zero
```

```python
x = float(input('Enter numerator: '))
y = float(input('Enter denominator: '))
if y == 0:
    print('Denominator cannot be zero.')
else:
    r = x / y
    print('r =',r)
```

Runtime error is fixed here.